



Venus Web API

Programmer's Manual

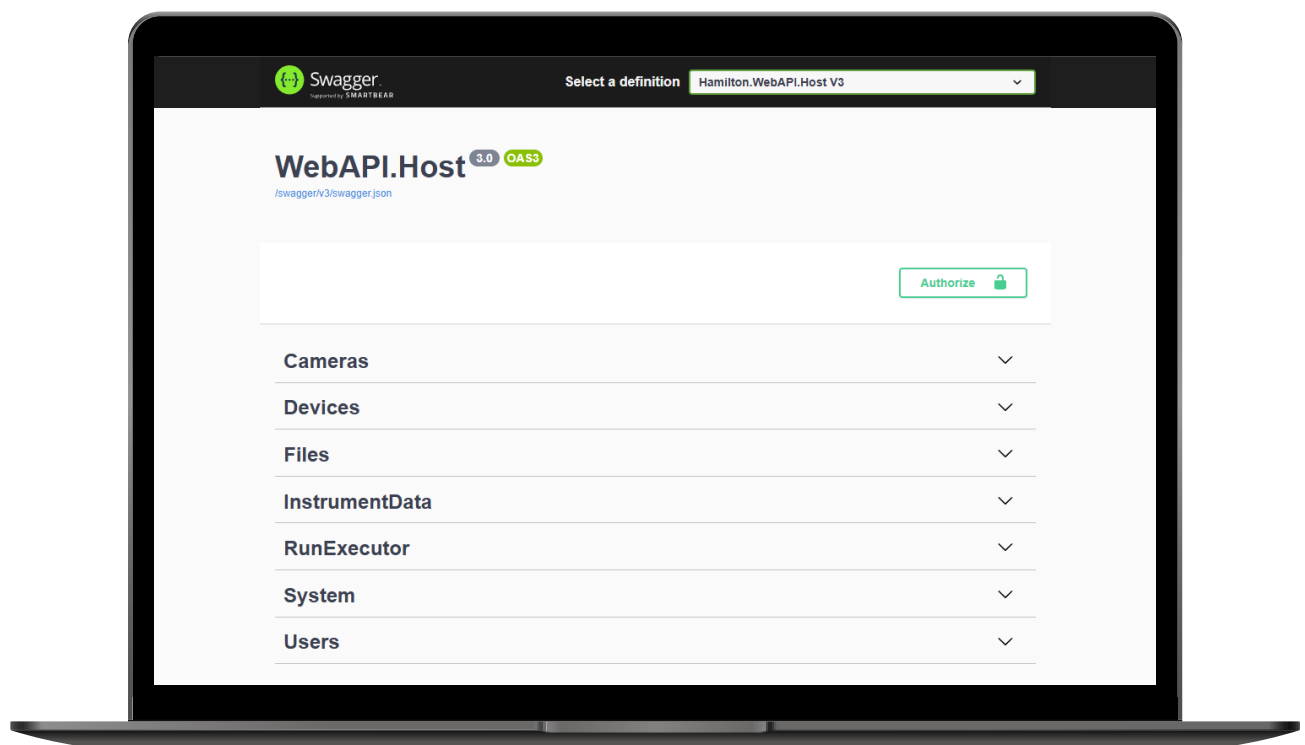


Table of Contents

Chapter 1: About	1
1.1 Introduction	2
1.2 About this Manual	2
 Chapter 2: Venus Web API	 3
2.1 Requirements	4
2.2 Installation	4
 Chapter 3: API Reference Documentation (Swagger)	 5
3.1 Enable Swagger for API Documentation	7
3.1.1 Categories	7
3.2 Enable Swagger for Identity Provider	8
3.2.1 Categories	8
3.3 Usage	9
 Chapter 4: SignalR	 11
4.1 About SignalR	12
4.2 SignalR Client Example	12
4.3 SignalR Push Notifications	14
4.3.1 Device Notifications	14
4.3.1.1 On Control Panel Complete	14
4.3.1.2 On Device Created	14
4.3.2 Labware Notifications	15
4.3.2.1 Labware Loaded State Changed	16
4.3.2.2 Labware Position State Changed	16
4.3.2.3 Labware State Changed	16

4.3.3	Runtime Notifications	16
4.3.3.1	New Runtime Error	16
4.3.3.2	New Runtime Status	16
5.3.3.3	New Runtime Trace	17
4.3.4	On Progress Changed	17
4.3.5	On Load Method	17
4.3.6	On Recovery Dialog	17
4.3.7	On DeckWatch Recording Export Started	18
Chapter 5:	Authentication and Authorization	19
5.1	Operating System Security Groups	21
5.2	Using External Identity and Access Management Providers	21
5.3	Customizing VENUS Web API Authentication and Authorization Settings	22
5.3.1	Customizing VENUS Web API Kestrel Settings	23
5.3.2	Customizing Identity Provider Configuration	23
Chapter 6:	Running Operations	25
6.1	How to use a Secured Endpoint	26
6.1.1	Viewing Current Authentication Mode	26
6.1.2	Enabling Authentication	27
6.1.3	Generating an Access Token	27
6.1.4	Using a Secured Endpoint	28
6.2	How to Run a Method	29
6.2.1	Load a Method	30
6.2.2	Start, Resume, Pause, and Abort a Method	31
6.2.3	Unload a Method	32
6.3	How to View Instrument State	32
6.3.1	Get Device ID from Runtime	32
6.3.2	Get the Deck Layout of a Device	33
6.3.3	Subscribing to Labware Events	34
6.3.3.1	LabwareStateChanged Subscription	34
6.3.3.2	LabwarePositionStateChanged Subscription	34
6.3.3.3	LabwareLoadedStateChanged Subscription	34
6.4	How to Get Detailed Run Data	35
6.4.1	Get Run GUID	35
6.4.2	Get Detailed Run Data	36
6.4.3	Get Current Run Data	36
6.5	How to Subscribe to SignalR Push Notifications	37

Chapter 7: Sample Clients	38
7.1 Angular Sample Client	39
7.1.1 Setup	40
7.1.2 Features	40
7.1.2.1 Authentication and Authorization	40
7.1.2.2 Executing Methods	41
7.1.3 Project Structure	42
7.2 React.JS Client	43
7.2.1 Setup	43
7.2.2 Features	44
7.2.2.1 Authentication and Authorization	44
7.2.2.2 Executing Methods	45
7.2.3 Project Structure	46

Chapter 1

About

- 1.1 Introduction
- 1.2 About this Manual

1.1 Introduction

The VENUS Web API provides a **RE**presentational **S**tate **T**ransfer (REST) web service and real-time web responsiveness via SignalR. The web service provides access to VENUS functions to run methods with their associated liquid handling system. When running methods, the Web API provides real-time notifications with a SignalR API for clients to subscribe to. With the Web API, developers can create custom desktop, mobile, or web applications to control VENUS RESTfully.

1.2 About this Manual

Version	Revision	Released	Description
01	A	03/2025	Document Created for API release 6.3

All efforts have been made to ensure the accuracy of the contents of this manual.

Hamilton Company can assume no responsibility for any errors in this manual or their consequences. If any errors are found, please contact Hamilton Company.

Reproduction of any part of this manual in any form whatsoever without the express written consent of Hamilton Company is forbidden.

The contents of this manual are subject to change without notice.

Copyright ©2025 Hamilton Company. All rights reserved.

VENUS® is a registered trademark of Hamilton Company.

VANTAGE Liquid Handling System® is a registered trademark of Hamilton Company.

For the latest revisions of Hamilton manuals, drivers, and software, contact Hamilton support.

Chapter 2

Venus Web API

2.1 Requirements

2.2 Installation

2.1 Requirements

Use of the VENUS Web API for development of custom applications requires familiarity with REST web services, JSON, and full-stack technologies, at a minimum.

When using the API for custom applications, it is recommended to have an engineer with experience in full-stack web development.

Some skills to include but are not limited to for development:

- HTML
- CSS
- JavaScript/TypeScript
- Server-Side technologies (e.g., ASP.NET, Node.JS, PHP, Python)

2.2 Installation

The VENUS Web API is installed with VENUS.

The VENUS Web API is packaged in the VENUS installation.

Chapter 3

API Reference Documentation (Swagger)

- 3.1 Enable Swagger for API Documentation
- 3.2 Enable Swagger for Identity Provider
- 3.3 Usage

API Reference Documentation (Swagger)

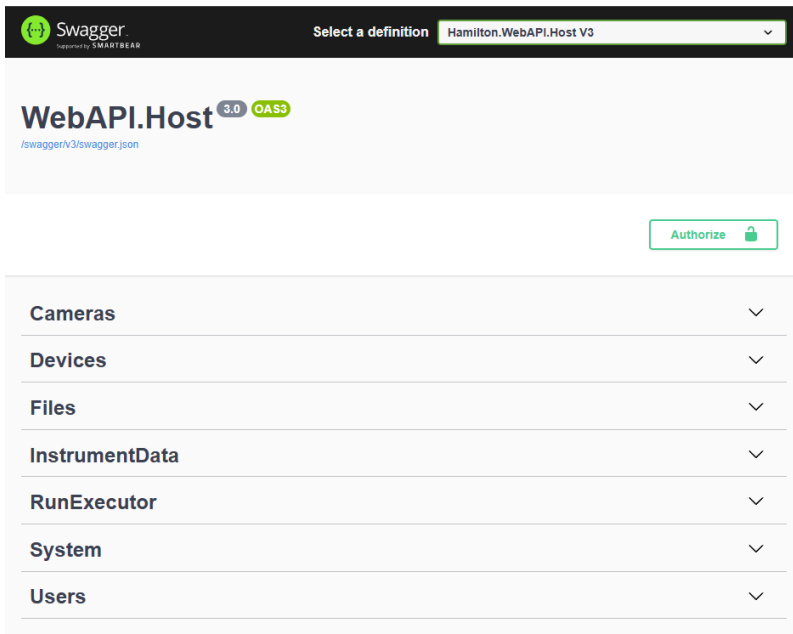


Figure 3-1: Viewing the API reference documentation through Swagger.

API reference documentation is provided for developers to explore and test the API.

When setup is complete, go to <http://localhost:51745/swagger/index.html> for the API documentation. Refer to [Section 3.1](#) “Enable Swagger for API Documentation” for setup instructions.

The API reference documentation contains several categories containing different endpoints and a schema section containing response schemas for the API endpoints.

To access different versions of the Web API, choose the desired version from the **Select a definition** drop-down at the top right of the page. The endpoints for the selected version are displayed.



Note: This documentation provides information on V3, which is the latest release of the Web API.

3.1 Enable Swagger for API Documentation

This section contains instructions on how to enable Swagger UI to view the API reference documentation.

1. Open C:\Program Files (x86)\Hamilton\Bin\WebAPI\WebAPI.Host.appsettings.json.
2. Locate the setting **swaggerUIEnabled** and replace “false” with “true”.
3. Save the file and close. The files are in a subdirectory in Program Files (x86) and are administrator-access only.
4. Open the **Services system application** and select “Hamilton VENUS WebAPI Host”.
5. Restart the Web API host.
6. Go to <http://localhost:51745/swagger/index.html> for the API reference documentation (Swagger UI).

3.1.1 Categories

Cameras	▼
Devices	▼
Files	▼
InstrumentData	▼
RunExecutor	▼
System	▼
Users	▼

Figure 3-2: Collapsed Categories found in the API Reference Documentation.

In Swagger, the API calls are separated into their respective categories for easier navigation. All endpoints shown in this section are from the **Hamilton.WebAPI.Host V3** definition. This section provides a list of the available categories and a general description of what the API endpoints can accomplish within their category.

- **Cameras:** Contains calls for retrieving camera information and performing recording operations.
- **Devices:** Contains calls that can:
 - Retrieve information from registered devices.
 - Retrieve instrument configuration.
 - Open the Control Panels for a requested device.

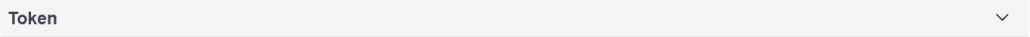
- **Files:** Contains calls related to method export, import, and validation. The category also contains endpoints to request files that are generated during a method run.
- **InstrumentData:** Contains calls to retrieve, update, delete, and save instrument data.
- **RunExecutor:** Contains calls that pertain to method loading and execution.
- **System:** Contains calls that request information about the VENUS software and its environment.
- **Users:** Contains calls that request information about the user.

3.2 Enable Swagger for Identity Provider

This section provides steps on how to enable Swagger for the Identity Provider service used for authentication and authorization.

1. Open C:\Program Files (x86)\Hamilton\Bin\IdentityProvider\IdentityProvider.appsettings.json.
2. Locate the setting **swaggerUIEnabled** and replace “false” with “true”.
3. Save the file and close. The files are in a subdirectory in Program Files (x86) and are administrator-access only.
4. Open the **Services system application** and select “Hamilton VENUS Identity Provider”.
5. Restart the Identity Provider service.
6. Go to <https://localhost:7229/swagger/index.html> for the Identity Provider reference documentation (Swagger UI).

3.2.1 Categories



Token

Figure 3-3: Token Category in the Identity Provider API documentation.

The identity provider service has a category for access tokens.

3.3 Usage

Swagger UI allows users to view and execute endpoints in the Web API and Identity Provider. If authentication is enabled, endpoints in V3 require a JSON Web Token (JWT) to make an authorized call to the endpoint.

Expand the endpoints to view their respective documentation on usage, parameters, headers, and request and response bodies.

To view the documentation, select the desired API endpoint.

If authentication is enabled, select “Authorize” to bring the authorization modal to view and insert the access token and select “Authorize” to authorize access for all endpoints.

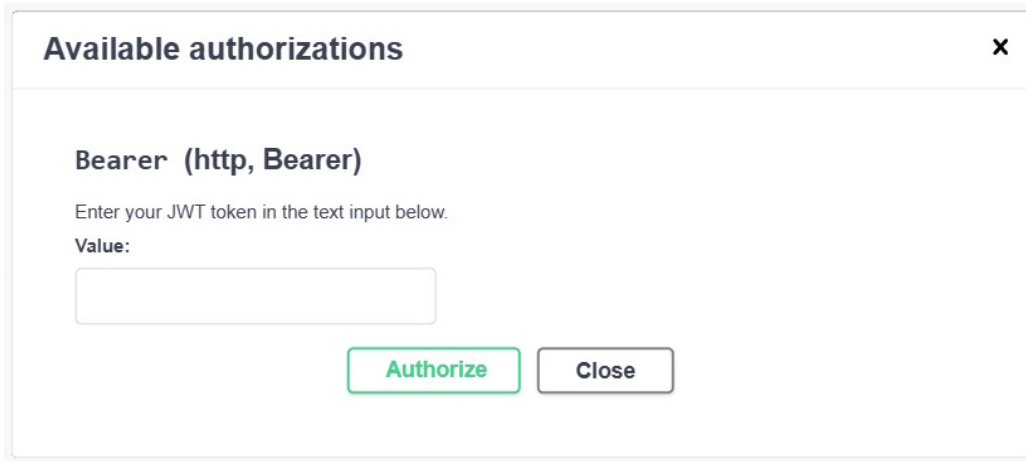
The image shows a modal window titled "Available authorizations" with a close button (X) in the top right corner. Inside the modal, the text "Bearer (http, Bearer)" is displayed. Below this, a message says "Enter your JWT token in the text input below." followed by the label "Value:". There is a text input field below the label. At the bottom of the modal, there are two buttons: "Authorize" (highlighted with a green border) and "Close".

Figure 3-4: Authorization modal.

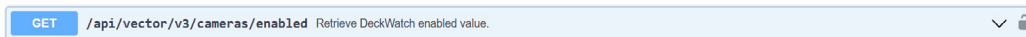
The image shows a single API endpoint entry in the Swagger UI. It consists of a blue button labeled "GET" followed by the endpoint path "/api/vector/v3/cameras/enabled". To the right of the path is a description: "Retrieve DeckWatch enabled value." On the far right, there is a dropdown arrow and a lock icon, indicating that the endpoint is secured.

Figure 3-5: Example of a secured, clickable API call.

To test the API endpoint:

1. Select “Try it out” to enable any parameter inputs and display the “Execute” button.
2. Populate the parameter inputs, if applicable, and select “Execute” to execute the endpoint.

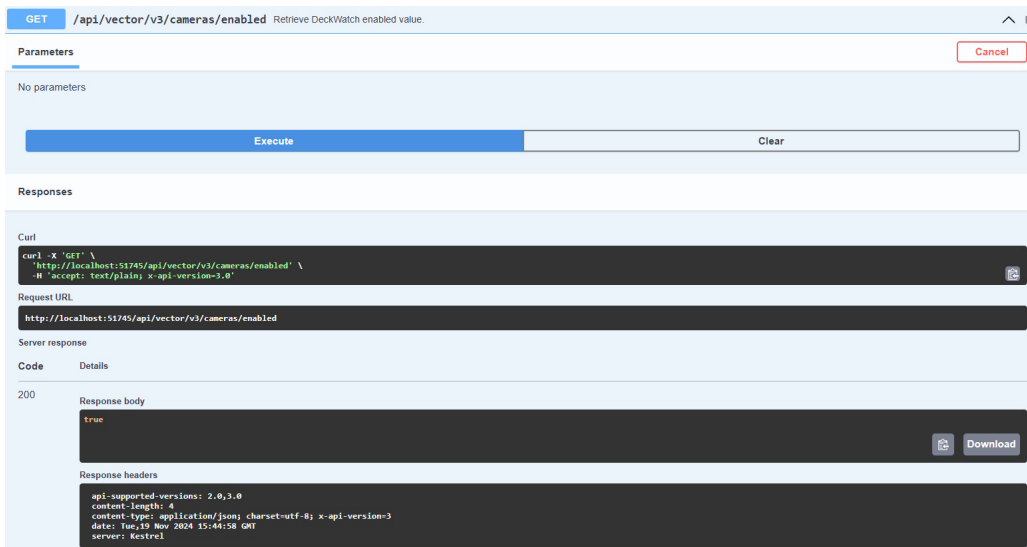


Figure 3-6: Example of a tested endpoint with a server response.

3. When executing the API endpoint, a server response returns a status code, response body, and response headers.

For more information about Swagger and its other developer tools, go to <https://swagger.io/>.

Chapter 4

SignalR

- 4.1 About SignalR
- 4.2 SignalR Client Example
- 4.3 SignalR Push Notifications

SignalR

This section describes how the VENUS Web API handles real-time events through a SignalR hub and how to set up a SignalR client.

The URL for the SignalR hub is:

<http://localhost:51745/signalR>.*

* The URL is meant to be used to create a connection to the SignalR Hub, not to be viewed in the browser.

4.1 About SignalR

SignalR provides real-time functionality to applications by providing bidirectional communication between clients and a hub. The VENUS Web API utilizes SignalR to provide real-time updates on instruments, traces, and more.

For more information about SignalR, go to <https://learn.microsoft.com/en-us/aspnet/signalr/overview/getting-started/introduction-to-signalr>

4.2 SignalR Client Example

This section provides a sample SignalR Client written in JavaScript. For JavaScript based clients, it is required to install the following dependency:

- @microsoft/signalr

For more information about the dependency, go to <https://www.npmjs.com/package/@microsoft/signalr>.

Below is a snippet of the sample code written in JavaScript for a SignalR Client.



Note: The sample code is for demo purposes only and should not be used in production.

```
import { HubConnectionBuilder, HttpTransportType } from "@microsoft/signalr";

const signalRUrl = `http://localhost:51745/signalR`;

//Building Signal-R Hub Connection
const hubConnection = new HubConnectionBuilder()
  .withUrl(signalRUrl, {
    skipNegotiation: true,
    withCredentials: false,
    transport: HttpTransportType.WebSockets,
  })
  .withAutomaticReconnect()
  .build();

//Subscription Example: Subscribe to push notification "NewRuntimeTrace"
hubConnection.on("NewRuntimeTrace", (data) => {
  console.log(data);
});

//Start Signal-R Hub Connection
hubConnection
  .start()
  .then(() => {
    console.log("Signal-R Hub connected.");
  })
  .catch(() => {
    console.log("Error connecting to Signal-R Hub.");
  });
```

Figure 4-1: Sample code for the SignalR connection.

The sample code provides a connection to the SignalR hub with the “hubConnection” variable and a handler for the “NewRuntimeTrace” push notification.

To run the sample code in a new project using npm (node package manager):

1. In the terminal of the desired project folder, execute “npm init” to initialize the project folder. After initializing the folder for a new project, a package.json file is created.
2. Install the SignalR library with the following command,
npm install @microsoft/signalr.
3. Create an MJS file and copy the sample client above into the file.
4. Run the code by executing node [filename].mjs in the terminal. If a connection is made to the Signal-R hub, the message “Signal-R Hub connected.” is displayed in the console. If a connection is not made, the message “Error connecting to Signal-R Hub.” will be displayed in the console.

4.3 SignalR Push Notifications

This section provides schemas for the different push notifications provided by the SignalR Hub.

4.3.1 Device Notifications

4.3.1.1 On Control Panel Complete

Payload: Object

Message Name: OnControlPanelComplete

Schema: ControlPanelArgs

Key	Type	Value	Description
DeviceId	Int	Int	ID for the associated device

4.3.1.2 On Device Created

Payload: Object

Message Name: OnDeviceCreated

Schema: DeviceArgs

Key	Type	Value	Description
DeviceId	Int	Int	ID for the associated device.
deviceName	String	String	Shortened name of the device, no spaces.
deviceDisplayName	String	String	Full display name of the device.
hasDeckLayout	Boolean	True or False	Flag for if the device has a deck layout.
Simulation	Boolean	True or False	Flag for if the device is in a simulation.
Model	String	String or Empty	Contains the file path for the device model found in \Vector\RunMaster\Graphic folder.

4.3.2 Labware Notifications

The following schemas are used in each of the labware state change notifications.

Schema: DeckLayoutArgs

Key	Type	Value	Description
DeviceId	Int	Int	Unique device ID.
LabwareId	String	String	Labware ID.
PositionId	String	String empty	Position ID. Empty if not specified.
Position Coordinates	Coordinate	Coordinate	Position Coordinates. Only valid if the PositionId is not empty.
State	Int	ActionType: { lwcatSelected=0, lwcatProcessing=1, lwcatMisc=2, lwcatError=3, lwcatProcessed=4, lwcatReset=5, lwcatInputRequired=6 } or LoadState: { lwclsUnloaded=0, lwclsLoaded=1, lwclsUnloading=2, lwclsLoading=3, lwclsRequestUnload=4, lwclsRequestLoad=5 }	Labware state. Note: The DeckLayoutArgs can only be of type, "ActionType" or "LoadState". Not both. See the labware state change notifications listed below and their associated state type.

Schema: Coordinate

Key	Type	Value	Description
X	Double	Millimeter	X coordinate.
Y	Double	Millimeter	Y coordinate.
Z	Double	Millimeter	Z coordinate.

4.3.2.1 Labware Loaded State Changed

Payload: Object

Message Name: LabwareLoadedStateChanged

Schema: Refer to “Deck Layout Args” in [Section 4.3.2](#) “Labware Notifications” (status type: LoadState)

4.3.2.2 Labware Position State Changed

Payload: Object

Message Name: LabwarePositionStateChanged

Schema: Refer to “Deck Layout Args” in [Section 4.3.2](#) “Labware Notifications” (status type: ActionType)

4.3.2.3 Labware State Changed

Payload: Object

Message Name: LabwareStateChanged

Schema: Refer to “Deck Layout Args” in [Section 4.3.2](#) “Labware Notifications” (status type: ActionType)

4.3.3 Runtime Notifications

4.3.3.1 New Runtime Error

Payload: String

Message Name: NewRuntimeError

Key	Type	Value	Description
runtimeError	String	String	Message containing the runtime error.

4.3.3.2 New Runtime Status

Payload: Object

Message Name: NewRuntimeStatus

Schema: RuntimeStatusArgs

Key	Type	Value	Description
ExecutorEvent	ExecutorEvent	Undefined: 0, Initialized: 1, InitializedFailed: 2, Terminated: 3, Aborted: 4, Paused: 5, Running: 6, Unloaded: 7	Executor Event type.
Message	String	String null empty	Status message to display.

4.3.3.3 New Runtime Trace

Payload: String

Message Name: NewRuntimeTrace

Key	Type	Value	Description
runtimeTrace	String	String	Message containing the runtime trace to display.

4.3.4 On Progress Changed

Payload: Object

Message Name: OnProgressChanged

Schema: ProgressChangedEventArgs

Key	Type	Value	Description
methodName	String	String	Name of the method associated with the progress change.
message	String	String	Message to display.
progressPercentage	Int	Int	Percentage of progress with the running method.
elapsedSeconds	Int	Int	Elapsed run time in seconds.
remainingSeconds	Int	Int	Total remaining seconds left of the method.
schedMode	SchedulingMode	None: 0, Activity Scheduling: 1, Action Scheduling: 2	Type of scheduling.

4.3.5 On Load Method

Payload: String

Message Name: OnLoadMethod

Key	Type	Value	Description
loadedMethod	String	String	Message to notify that a method has been loaded.

4.3.6 On Recovery Dialog

Payload: Int

Message Name: OnRecoveryDialogDisplayed

Key	Type	Value	Description
DeviceId	Int	Int	Device ID associated with the recovery dialog.

4.3.7 On DeckWatch Recording Export Started

Payload: Object

Message Name: OnDeckWatchRecordingExportStarted

Key	Type	Value	Description
FileName	String	String	Filename of the recording.
StartTime	DateTime	DateTime	Start timestamp of the recording.
EndTime	DateTime	DateTime	End timestamp of the recording.
CameraIds	List<String>	List<String>	Collection of camera ids.

Chapter 5

Authentication and Authorization

- 5.1 Operating System Security Groups
- 5.2 Using External Identity and Access Management Providers
- 5.3 Customizing VENUS Web API Authentication and Authorization Settings

Authentication and Authorization

The VENUS Web API utilizes authentication and authorization in the 6.3 release. The use of authentication and authorization provides a secure way to interact with the VENUS Web API by ensuring that endpoints are only accessed by authorized users.

If function protection is on, a JWT is required by the Hamilton Identity Provider to be added to the request for the endpoint.

Authentication and Authorization is enabled with the Function Protection value in the system configuration.

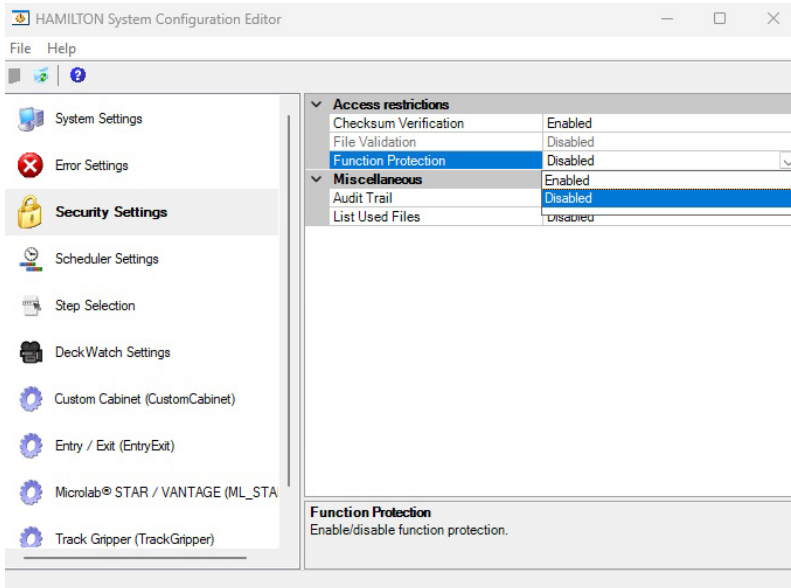


Figure 5-1: Function Protection setting in the system configuration editor.

Refer to [Section 6.1.2](#) “Enabling Authentication” on how to enable authentication for endpoints.

5.1 Operating System Security Groups

Upon installation of VENUS, new OS security groups are created to govern user access to functions in the VENUS software.

Table 5-1 shows the OS security groups created during VENUS installation. The roles are listed from the highest role precedence to the lowest.

Table 5-1: Operating system security groups created by VENUS

Group	User	Description	Role
Lab Service	Service Tech/ Lab Manager	All-access.	Admin
Lab Method Programmer	Method Programmer/ Lab Manager	Modify method and labware definitions.	Lab Method Programmer
Lab Operator 2	Routine user+	Run methods and rearrange deck.	Lab Operator 2
Lab Operator	Routine user	Run verified methods.	Lab Operator

Endpoints in the VENUS Web API are accessible and viewable by users of any VENUS role. An exception to this is the GET authentication-mode endpoint, which is accessible by users without a VENUS role. However, endpoints that handle sensitive data, such as log files, run history, and importing methods, are restricted to users with an Admin or Lab Method Programmer role.

Table 5-2 lists endpoints that require elevated permissions. Endpoints that are not listed in the table are accessible by users of any role.

Table 5-2: Endpoints that require elevated permissions

Endpoint	Permission
/api/vector/v3/files/import-method	Lab Service, Lab Method Programmer
/api/vector/v3/files/import-methods	Lab Service, Lab Method Programmer
/api/vector/v3/files/import-method-package	Lab Service, Lab Method Programmer
/api/vector/v3/files/cleanup	Lab Service, Lab Method Programmer
/api/vector/v3/system/venus/archive	Lab Service, Lab Method Programmer
/api/vector/v3/system/invalidate-run-history-cache	Lab Service, Lab Method Programmer

5.2 Using External Identity and Access Management Providers

The VENUS Web API supports the use of Identity and Access Management (IAM) solutions such as Auth0, Microsoft Entra, and Okta. An IAM solution can be used in place of the Identity Provider created by VENUS.

Before connecting an IAM solution to the VENUS Web API, users will need the following in their IAM solution:

- An API endpoint to request and validate access tokens which provide authorized access to the Web API.
- An application to authenticate users. This would be a client application interacting with the Web API.
- Roles matching the operating system security groups created by VENUS. View [Section 5.1](#) “Operating System Security Groups” for the list of available roles.
- Users with assigned VENUS roles.
- A role claim with “hamiltonRoles” as the claim name.

To configure the Web API to use an IAM for authentication and authorization:

1. Open C:\Program Files (x86)\Hamilton\Bin\WebAPI\WebAPI.Host.appsettings.json.
2. Locate the sub section **Bearer** in the **Authentication** section and update the following settings:
 - a. **Authority** to the base URL of the identity provider.
This is used to validate the issuer of the token.
 - b. **MetadataAddress** to the OpenID Connect metadata document of the associated issuer.
 - c. **ValidAudiences** to the list of valid audiences for the token.
 - d. **ValidIssuer** to the expected issuer of the token.
5. Save the file and close. The files are in a subdirectory in Program Files (x86) and are administrator-access only.
6. Open the **Services system application** and select “Hamilton VENUS WebAPI Host”.
7. Restart the Web API host.

5.3 Customizing VENUS Web API Authentication and Authorization Settings

The VENUS Web API provides settings that allow users to customize the Kestrel configuration and Identity Provider configuration.

VENUS Web API and Identity Provider uses the default host, Microsoft’s Kestrel server to self-host. For more information on Microsoft’s Kestrel server, go to <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/servers/kestrel?view=aspnetcore-9.0>.

5.3.1 Customizing VENUS Web API Kestrel Settings

The VENUS Web API can be configured to use HTTPS. If configured, the Kestrel configuration can also be customized to use a different hostname, certificate, and password.

To enable HTTPS with the Web API:

1. Open C:\Program Files (x86)\Hamilton\Bin\WebAPI\WebAPI.Host.appsettings.json.
2. Locate and update the setting **UseHttps** and replace “false” with “true”.
3. Save the file and close. The files are in a subdirectory in Program Files (x86) and are administrator-access only.
4. Open the **Services system application** and select “Hamilton VENUS WebAPI Host”.
5. Restart the Web API host.

To change the Web API’s Kestrel configuration:

1. Open C:\Program Files (x86)\Hamilton\Bin\WebAPI\WebAPI.Host.appsettings.json.
2. Locate and update the **Port**, **Path**, and **Password** settings to the desired values.
3. Save the file and close. The files are in a subdirectory in Program Files (x86) and are administrator-access only.
4. Open the **Services system application** and select “Hamilton VENUS WebAPI Host”.
5. Restart the Web API host.



Note: If the client changes the hostname, they will need to provide their own pfx certificate file, password and an updated DNS name saved in the bin directory C:\Program Files (x86)\Hamilton\Bin\.

5.3.2 Customizing Identity Provider Configuration

The Identity Provider that VENUS provides can use different client and Kestrel configurations. Client configuration is handled in the IdentityProvider.appsettings.json file. Whereas the Kestrel configuration is handled in the IdentityProvider.appsettings.Production.json file.

All files are in C:\Program Files (x86)\Hamilton\Bin\IdentityProvider\.

To change the Identity Provider's client configuration:

1. Open C:\Program Files (x86)\Hamilton\Bin\IdentityProvider\IdentityProvider.appsettings.json.
2. Locate the section **IdentityProvider** and update the settings **ClientID** and **ClientSecret** to the desired values.
3. Save the file and close. The files are in a subdirectory in Program Files (x86) and are administrator-access only.
4. Open the **Services system application** and select "Hamilton VENUS Identity Provider".
5. Restart the Identity Provider service.

To change the Identity Provider's Kestrel configuration:

1. Open C:\Program Files (x86)\Hamilton\Bin\IdentityProvider\IdentityProvider.appsettings.json.
2. Locate the sub section **HttpsPFXFile** under the **Kestrel** section and update the settings **URL**, **Path**, and **Password** to the desired values.
3. Save the file and close. The files are in a subdirectory in Program Files (x86) and are administrator-access only.
4. Open the **Services system application** and select "Hamilton VENUS Identity Provider".
5. Restart the Identity Provider service.



Note: When changing the Kestrel configuration, the URL protocol must be HTTPS. The host is explicitly wildcarded. If the client changes the hostname, they will need to provide their own pfx certificate file, password and an updated DNS name saved in the bin directory C:\Program Files (x86)\Hamilton\Bin\.

Chapter 6

Running Operations

- 6.1 How to use a Secured Endpoint
- 6.2 How to Run a Method
- 6.3 How to View Current Instrument State
- 6.4 How to Get Detailed Run Data
- 6.5 How to Subscribe to SignalR Push Notifications

Running Operations

This section describes how to combine multiple API calls to achieve a specific operation.

6.1 How to use a Secured Endpoint

This section contains instructions on how to use a secured endpoint if authentication is enabled.

To use secured endpoints, it is necessary to view the current Authentication Mode to identify whether the endpoints are secured. By default, authentication is disabled on installation of VENUS, however the Authentication Mode can be checked by making a call to an endpoint. If the Authentication Mode is set to “None”, users can enable authentication through the function protection setting in the system configuration. Once endpoints are secured, calls to endpoints require an access token. Access tokens are accessed through an endpoint call to the Hamilton identity provider.

6.1.1 Viewing Current Authentication Mode

This section describes how to view the current Authentication Mode by calling an endpoint.

For VENUS, there are two modes of authentication:

- None
- Windows AD

To view the current authentication mode, a call to the GET `v3/system/authentication-mode` endpoint is required.

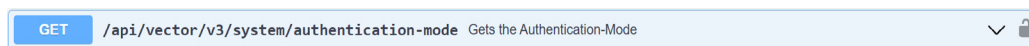


Figure 6-1: Authentication mode endpoint.

If access to the endpoint is denied, refer to [Section 6.1.4](#) “Using a Secured Endpoint”.

Upon a successful call to the endpoint, the API provides a response containing the authentication mode. The response body contains “None” or “Windows AD”.

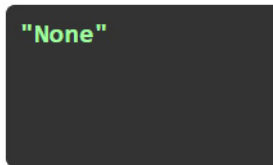


Figure 6-2: Example of the authentication mode endpoint response body.

6.1.2 Enabling Authentication

This section describes how to enable authentication through the **Hamilton Configuration Editor** application.



Note: Authentication can only be enabled by users in the “Lab Service” group.

Before enabling authentication, assign users to the appropriate operating system security group. Refer to [Section 5.1](#) “Operating System Security Groups” for more information.

To enable authentication:

1. Open **Hamilton.HxConfigEditor**.
2. Select the **Security Settings** tab.
3. Under **Access restrictions**, select the dropdown for **Function Protection** and select “enabled”.
4. Save the settings.
5. Close **Hamilton.HxConfigEditor**.
6. Open the **Services system application** and select “Hamilton VENUS WebAPI Host”.
7. Restart the Web API host.

6.1.3 Generating an Access Token

This section provides steps on how to obtain an access token through the identity provider.

To obtain an access token, a call must be made to the token endpoint in the identity provider.



Figure 6-3: Authorization token endpoint.

The POST `/v1/authorization/token` endpoint requires the following in the request body: the OAuth grant type, the user's username, the user's password, the client's ID, the client's secret, an optional OAuth scope.

Figure 6-4: Authorization token endpoint's request body.

Default values are provided for the **grant_type** and **scope**. These default values can be used to execute the endpoint along with valid credentials to an account tied to VENUS operating system security groups.

After a successful call to the endpoint, the endpoint returns a response body containing the access token.

```
{
  "access_token": "string",
  "token_type": "string",
  "expires_in": 0,
  "id_token": "string"
}
```

Figure 6-5: Authorization token endpoint's response body.

6.1.4 Using a Secured Endpoint

This section provides steps on how to use a secured endpoint when authentication is enabled for the Web API.

Figure 6-6: Example of a secured endpoint.

By default, VENUS sets the authentication mode to “None”. Resulting in unrestricted access to V3 endpoints. To restrict access to endpoints, refer to [Section 6.1.2 “Enabling Authentication”](#) for steps on how to secure endpoints.

To execute a secure endpoint, an access token is required to make the call. Refer to [Section 6.1.3](#) “Generating an Access Token” for steps on how to obtain an access token for a user. The access token is used as a bearer token in the authorization header for the endpoint.

An access token can be used for a secured endpoint in an HTTPS request or through Swagger. When using Swagger, refer to [Section 3.3](#) “Usage” on how to apply the access token to gain access to secured endpoints. For HTTPS requests, the access token can be applied by adding an authorization header with authentication scheme “Bearer” and the access token in the credential’s parameter.

If an incorrect access token is provided, the endpoint provides a response with an unauthorized error message.

```
{
  "Message": "An error occurred while processing your request.",
  "Errors": {
    "ErrorMessage": [
      "401 Unauthorized"
    ]
  }
}
```

Figure 6-7: Example response body of an unauthorized request.

If the correct access token is provided, the endpoint returns the expected response for the associated endpoint.

6.2 How to Run a Method

This section describes how to run subsequent endpoint calls to run a method.



Note: This section applies when the authentication mode is “None”. If authentication is enabled, refer to [Section 6.1.4](#) “Using a Secured Endpoint” on how to make an authorized request.

The main parts of running a method are loading, executing, and unloading. Before a user can execute a start, stop, or abort call, the method must be loaded using the load endpoint in the Web API. After the method is loaded, the user can perform start, stop, or abort calls for the method. After method execution has completed, or when the instrument state allows, the user can unload the method.

6.2.1 Load a Method

This section describes how to use the API to load a method.

A method can be loaded via the load PUT /execution/load endpoint operation found in the RunExecutor category with the file path, simulation flag, and parent window handle.

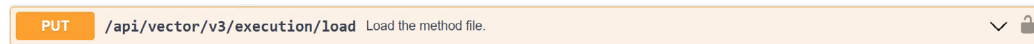


Figure 6-8: Load Method endpoint in Swagger.

```
{
  "filePath": "string",
  "simulation": true,
  "parentWindowHandle": 0
}
```

Figure 6-9: Example JSON of the request body for the Load PUT operation.

The filePath parameter requires the full path of the .hsl file of the desired method. For example, C:\\Methods\\TestMethod.hsl would be sufficient in making the request, whereas \\Methods\\TestMethod.hsl would not.

The simulation parameter allows developers to trigger a simulation of the method associated with the filePath parameter. All devices associated with the method are also simulated.

The **parentWindowHandle** parameter requires the native window handle ID (HWND) of the application's window. VENUS uses the HWND to place dialogs on top of the main application window. If an incorrect HWND is passed into the parameter, the method is aborted during execution. If the provided HWND for the application is 0, the dialogs could appear in the background.

If a device is created when loading a method, the SignalR API pushes an "OnDeviceCreated" notification, where its payload contains information for the device.

When a load operation is complete, the SignalR API pushes a notification of **NewRuntimeStatus** bearing a payload of either "Initialized" or "Initialize Failed". The Web API provides a response query with a status code, header, and response body.

```
{
  "result": true,
  "message": "string",
  "resetRunExec": true
}
```

Figure 6-10: Example JSON of the response body from a test execution of the Load PUT operation.

6.2.2 Start, Resume, Pause, and Abort a Method

This section describes how to start, resume, pause, and abort a method. When a method is loaded and the instrument status is “Initialized”, the user can call the start endpoint to execute the method. To obtain instrument status, refer to the instructions in [Section 6.3](#) “How to View Current Instrument State”. The endpoint for the start method is POST/execution/start.



Figure 6-11: Start endpoint in Swagger.

When starting a method, the instrument status switches from “Initialized” to “Running”. While the method is running, and if the client is subscribed to the push notifications the SignalR API hub provides real-time updates. The various SignalR push notifications that are provided to a subscribed client are “[On Progress Changed](#)”, “[New Runtime Status](#)”, and “[New Runtime Trace](#)”. For more information on how to subscribe to push notifications, refer to [Section 6.4.3](#) “Get Current Run Data”.

While the method is running, the client can execute the abort at endpoint POST/execution/abort and pause at endpoint POST/execution/pause.



Figure 6-12: Abort endpoint in Swagger.

The abort endpoint can be called when the instrument state is “Running” or “Paused”. If the abort endpoint is called at any other instrument state, the method will not be aborted, and the Web API will provide an error message stating an invalid request has been made.



Figure 6-13: Pause endpoint in Swagger.

When a method is paused, the client can execute the resume endpoint POST/execution/resume. If the abort endpoint is called at any other instrument state, the method will not resume, and the Web API will provide an error message stating an invalid request has been made.



Figure 6-14: Resume endpoint in Swagger.

When the method is completed, the instrument status is posted as “Terminated”, allowing the client to execute the unload method endpoint.

6.2.3 Unload a Method

This section describes how to unload a method that is currently loaded. For an aborted, initialized, or terminated status, the currently loaded method can be unloaded. The unload method endpoint is POST/execution/unload-method.

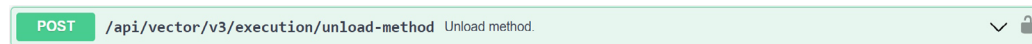


Figure 6-15: Unload endpoint in Swagger.



Important: Do not unload a method when the instrument is in a running or paused state.

6.3 How to View Instrument State

This section describes how to view the instrument's current state.



Note: This section applies when the authentication mode is "None". If authentication is enabled, refer to [Section 6.1.4](#) "Using a Secured Endpoint" on how to make an authorized request.

When gathering information about the instrument, the device ID must be obtained during runtime. The device ID is used to retrieve information such as instrument configuration and deck layout. To obtain information on labware equipment, clients can subscribe to one or all of the labware state change notifications provided by the SignalR Hub.

6.3.1 Get Device ID from Runtime

After loading a method that contains a device(s), obtain the device ID of the device(s) to obtain the instrument configuration, and obtain the deck layout of the device.

Obtain the device ID by either a SignalR notification or the Web API. To obtain the device ID from the SignalR Hub, subscribe to the "OnDeviceCreated" notification shown in [Section 4.3.1.2](#), and parse the response to obtain the device ID. To obtain the device ID through the Web API, execute the endpoint, GET /devices/runtime, and parse the response for the device ID.

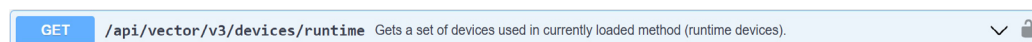


Figure 6-16: Runtime devices endpoint from Swagger.

When executing the endpoint, the API provides a response body containing device information.

```
{
  "deviceName": "string",
  "deviceDisplayName": "string",
  "imageFile": "string",
  "hasDeckLayout": true,
  "deviceId": 0,
  "simulation": true,
  "controlPanelSupport": 0,
  "windowHandle": 0
}
```

Figure 6-17: Runtime response body.

6.3.2 Get the Deck Layout of a Device

This section describes how to get the deck layout of a device used in a method. The deck layout of a device can be retrieved via a GET endpoint in the **Devices** category. The endpoint used for the method is GET/devices/deck-layout/[deviceId].

GET /api/vector/v3/devices/deck-layout/{deviceId} Get deck layout for the loaded method.

Figure 6-18: Device deck layout endpoint in Swagger.

During execution of the GET endpoint, the deck layout is returned. The deck layout contains the deck, labware, and deck layers information. See Figure 6-19 for an example snippet of the schema for the returned deck layout.

```
{
  "deck": {
    "id": "string",
    "parentId": "string",
    "anchor": {
      "x": 0,
      "y": 0,
      "z": 0
    },
    "dimension": {
      "x": 0,
      "y": 0,
      "z": 0
    },
    "rotation": 0,
    "model": "string",
    "modelOffset": {
      "x": 0,
      "y": 0,
      "z": 0
    },
    "bitmap": "string",
    "visible": true,
    "originOffset": {
      "x": 0,
      "y": 0,
      "z": 0
    }
  }
}
```

Figure 6-19: Deck layout response snippet from Swagger.

6.3.3 Subscribing to Labware Events

This section describes how to subscribe to the labware push notifications and how to use the information associated with labware state changes.

6.3.3.1 LabwareStateChanged Subscription

For the **LabwareStateChanged** push notification, the event provides the client a payload that contains the action state of the labware. To subscribe, provide the notification name **LabwareStateChanged** in the example code snippet provided in [Section 6.5](#) “How to Subscribe to SignalR Push Notifications”. For the push notification’s response data schema, refer to [Section 4.3.2.3](#) “Labware State Changed”. The state can indicate that the labware is selected, processing, error, process, reset, requires input, or a miscellaneous state.

6.3.3.2 LabwarePositionStateChanged Subscription

For the **LabwareLoadedStateChanged** push notification, the event provides the client a payload that contains the load state of the labware. To subscribe, provide the notification name **LabwareLoadedStateChanged** in the example code snippet provided in [Section 6.5](#) “How to Subscribe to SignalR Push Notifications”. For push notification’s response data schema, refer to [Section 4.3.2.1](#) “Labware Loaded State Changed”. The state can indicate that the labware is **unloaded, loaded, unloading, loading, requesting unload, or requesting load**.

6.3.3.3 LabwareLoadedStateChanged Subscription

For the **LabwarePositionStateChanged** push notification, the event provides the client a payload that contains the position change of specified wells or containers in a plate in the labware like carrier. To subscribe, provide the notification name **LabwarePositionStateChanged** in the example code snippet provided in [Section 6.5](#) “How to Subscribe to SignalR Push Notifications”. For the push notification’s response data schema, refer to [Section 4.3.2.2](#) “Labware Position State Changed”. The payload contains the position ID and the coordinate change of the labware.

6.4 How to Get Detailed Run Data

This section describes how to obtain detailed data of a run.



Note: This section applies when the authentication mode is "None". If authentication is enabled, refer to [Section 6.1.4](#) "Using a Secured Endpoint" on how to make an authorized request.

To gather detailed run data, the run GUID must be obtained first.

6.4.1 Get Run GUID

The **GET run GUID** is obtained through a call to an endpoint in the file category. This section demonstrates how to use the endpoint in the file category.

The endpoint in the file category, GET files/runtime-data is used to obtain the run GUID and other runtime data of previous runs. When using that endpoint, there are optional filtering parameters that can be passed into the endpoint's parameters to filter through runtime data files in a specific time frame. Otherwise, the endpoint returns all runtime data files that are currently stored.

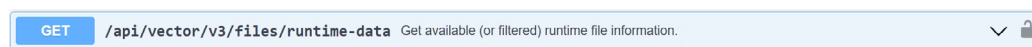


Figure 6-20: Runtime data endpoint.

```
{
  "files": [
    {
      "methodName": "string",
      "dbFileName": "string",
      "runGuid": "string",
      "startTime": "2024-11-19T16:10:55.265Z",
      "endTime": "2024-11-19T16:10:55.265Z",
      "computerName": "string",
      "runState": "string",
      "duration": 0
    }
  ]
}
```

Figure 6-21: Example JSON of the response body for the runtime data GET endpoint.

6.4.2 Get Detailed Run Data

This section describes how to get detailed run data. To obtain the detailed run data, obtain the run GUID with the methods provided in [Section 6.4.1](#) “Get Run GUID”.

When a **run GUID** is obtained for the desired run, the run GUID is used as a parameter for the endpoint, GET files/detailed-run-data/{runtimeGUID}. After the endpoint has been successfully executed, the server responds with a response body containing the detailed run data.



Figure 6-22: Detailed run data endpoint.

```
{
  "methodName": "string",
  "dbFileName": "string",
  "runGuid": "string",
  "startTime": "2024-11-19T16:11:50.101Z",
  "endTime": "2024-11-19T16:11:50.101Z",
  "computerName": "string",
  "runState": "string",
  "duration": 0,
  "userName": "string",
  "instruments": [
    {
      "instrumentName": "string",
      "instrumentSerialNumber": "string",
      "instrumentSimulation": true,
      "softwareVersion": "string"
    }
  ]
}
```

Figure 6-23: Example JSON of the response body for the detailed run data endpoint.

6.4.3 Get Current Run Data

This section describes how to gather run data during the runtime of a method. To obtain run data of a method that is currently running, a call to the endpoint GET execution/current-run-data is required.

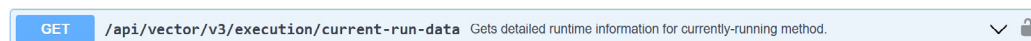


Figure 6-24: Current run data endpoint.

To call the endpoint, a method must be loaded and running to successfully obtain current run data. Refer to section [Section 6.2](#) “How to Run a Method” for steps on how to load and run a method.

After successful execution, the endpoint returns a response body of JSON format with the detailed run data. If the execution of the endpoint is unsuccessful, the API will provide an error message stating that the detailed runtime info is not available.

```
{
  "methodName": "string",
  "dbFileName": "string",
  "runGuid": "string",
  "startTime": "2024-11-19T16:12:55.274Z",
  "endTime": "2024-11-19T16:12:55.274Z",
  "computerName": "string",
  "runState": "string",
  "duration": 0,
  "userName": "string",
  "instruments": [
    {
      "instrumentName": "string",
      "instrumentSerialNumber": "string",
      "instrumentSimulation": true,
      "softwareVersion": "string"
    }
  ]
}
```

Figure 6-25: Example JSON of the response body for the current run data endpoint.

6.5 How to Subscribe to SignalR Push Notifications

To subscribe to a SignalR push notification(s), a sample client, similar to the example in [Section 4.2](#) “SignalR Client Example”, is required. To create a subscription, create a hub connection to the SignalR hub’s URL, <http://localhost:51745/signalR> with a WebSocket transport. After the hub connection is created, a handler can be registered to handle a specific push notification. The following is an example of how to register a handler for a push notification in TypeScript:

```
Sthis.hubConnection.on(
  'On-Push-Notification',
  (pushNotification: PushNotification) => {
    console.log(pushNotification);
  }
);
```

Figure 6-26: Push notification code.



Note: The code is for demo purposes only and should not be used in production.



Note: Any words using “push notification” are placeholder text to be replaced by any of the push notifications listed in [Section 4.3](#) “SignalR Push Notifications”.



Note: **On-Push-Notification** is the push notification name, “pushNotification” is the data returned by the hub, and “PushNotification” is the push notification’s schema. When using JavaScript, omit “PushNotification”.

Chapter 8

Sample Clients

8.1 Angular Sample Client

8.2 React.JS Client

Sample Clients

This section provides information of sample clients using the VENUS Web API. The primary intent of the clients is to demonstrate how to develop with the API.

To run the methods provided in this sample client, ensure that the VANTAGE instrument is included on installation and the “Checksum Verification” setting in the **Hamilton System Configuration Editor**, is set to “Disabled”.



Note: These sample clients are for demonstration purposes only. Do not use in production.

7.1 Angular Sample Client

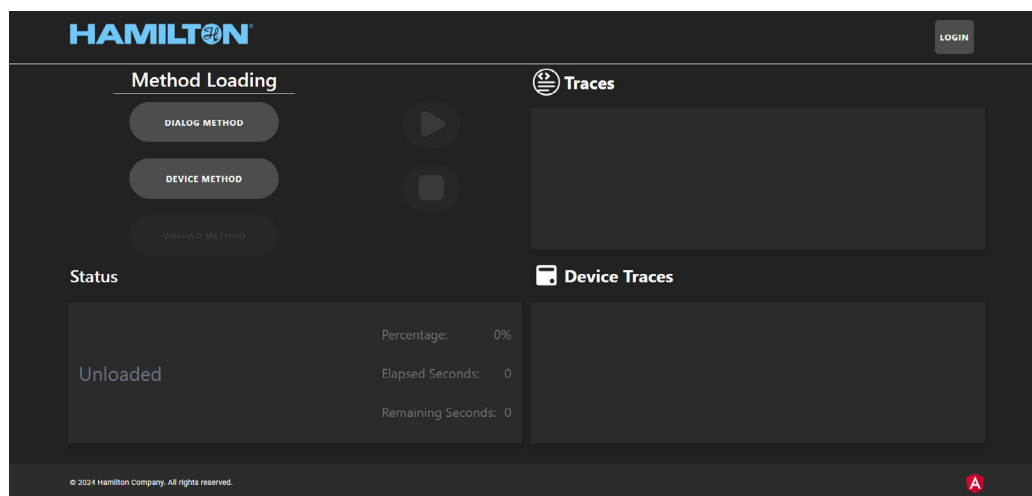


Figure 7-1: Snapshot of the home page.

The Angular sample client is built with the Angular web framework and an Express.JS backend application to handle server-side functions.

For more information about the Angular sample client, refer to the README.md markdown file in C:\ProgramFiles (x86)\Hamilton\Bin\WebAPI\samples\Angular.

7.1.1 Setup

This section provides instructions on how to set up an environment to run the Angular Client. The instructions provided are also included in the README.md guide under the project's source files.



Note: To run the example client, a set of pre-requisites are required. Refer to the markdown file in the project in C:\Program Files (x86)\Hamilton\Bin\WebAPI\samples\Angular to view the pre-requisites.

To launch the example client:

1. In a terminal or IDE, navigate to the directory of the Angular sample client:
C:\Program Files (x86)\Hamilton\Bin\WebAPI\Samples\Angular
2. Install the dependencies by running npm install.
3. Run npm start.

7.1.2 Features

This section provides descriptions of the features provided in the sample client.

7.1.2.1 Authentication and Authorization

If authentication on the machine is enabled, users are required to log in to an authorized account to perform calls to secured endpoints. The example client provides a feature to allow users to log in to their authorized account. For more information on secured endpoints and how to use them, refer to [Section 6.1](#) “How to use a Secured Endpoint”.

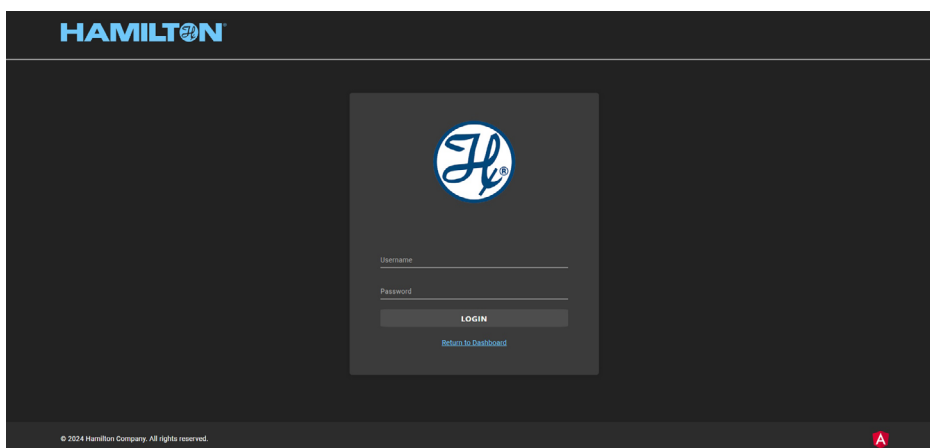


Figure 7-2: Snapshot of the Login page.

When starting the application, if the user is not signed in and authentication is enabled, the application launches on the Login page. Otherwise, it launches on the Home page.

When authentication is enabled, users must log in with the credentials of an authorized OS security group account to authenticate with the identity provider and gain authorized access to the dashboard executables and other functions in the VENUS software.

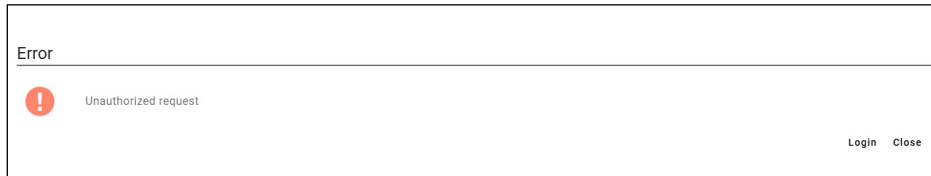


Figure 7-3: Snapshot of an error dialog for an unauthorized request.



Note: Refer to [Section 5.1](#) “Operating System Security Groups” for information on the security groups used to grant access to functions in the VENUS software.

7.1.2.2 Executing Methods

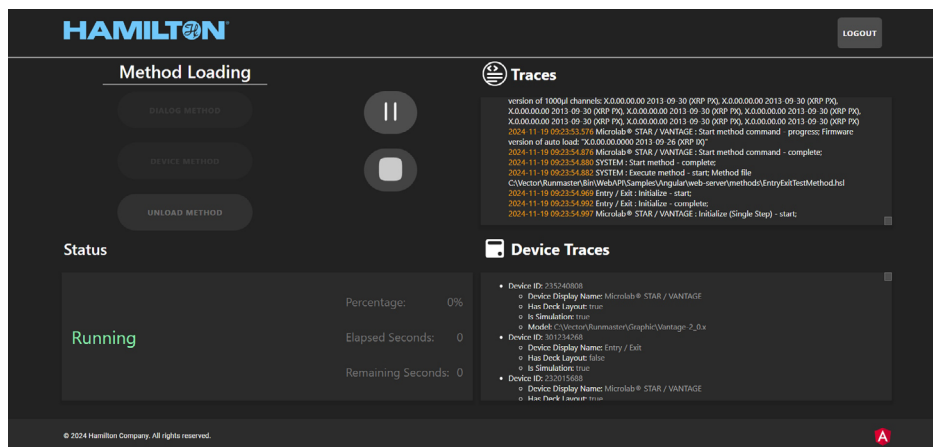


Figure 7-4: Snapshot of the sample client running the device method.

The sample client provides two methods that can be executed. One method provides a dialog prompt to the user, and the other method provides a simulated initialization procedure for an instrument.

When a method is running, the sample client updates the instrument’s status, runtime traces, and if the method uses a device, device traces. Refer to [Section 6.2](#) “How to Run a method” for more information on how to run a method and the operations that are available during runtime.

7.1.3 Project Structure

Name	Description
src/	Source files for the root-level application.
src/app	Folder containing the project's logic and data.
src/app/core/components/app-footer	Footer component used in the application.
src/app/core/components/app-header	Header component used in the application.
src/app/core/interceptors/	Interceptors for authorization and errors.
src/app/core/pages/home	Home page of the application.
src/app/core/pages/login	Login page of the application.
src/app/core/services/	Services for authentication, local storage, session storage, and user session management.
src/app/modules/material-ui	Module for material-ui components.
src/app/modules/svgicons	Module for registering SVG icons with material-ui.
src/app/shared/components/auth-button	Button component for logging in and out.
src/app/shared/components/error-modal	Modal component for errors.
src/app/shared/components/method-dashboard	Parent component containing run-executor components.
src/app/shared/components/runexecutor-components/abort-button	Button component for aborting a method.
src/app/shared/components/runexecutor-components/load-method	Button component for loading a method.
src/app/shared/components/runexecutor-components/pause-play-toggle	Button component for pausing, playing, and resuming methods.
src/app/shared/components/runexecutor-components/unload-method-button	Button component for unloading a method.
src/app/shared/components/spinner	Component for a spinner to indicate loading.
src/app/shared/components/status-window	Window component for displaying instrument status.
src/app/shared/components/trace-components/device-window	Trace window component for displaying device traces.
src/app/shared/components/trace-components/trace-window	Trace window component for displaying runtime traces.
src/app/shared/pipes	Angular Pipe for sanitizing html.
src/app/shared/services/	Shared services for: error modals, accessing VENUS functions, SignalR, application startup checks.
src/assets	Contains the applications svg icons and images.
src/environments	Environment variables for the application.
web-server/	Express.js app with a C++ node addon.
web-server/build	Pre-compiled build of the node addon.
web-server/methods	VENUS instrument methods.
Angular-Client-Architecture.png	Image of the application's architecture.
angular.json	Workspace configuration.
package-lock.json	NPM dependency tree.

Name	Description
package.json	NPM dependency list.
README.md	Workspace documentation.
server.ts	Angular's express app.
tailwind.config.js	TailwindCSS configuration.
tsconfig.app.json	TypeScript application compiler.
tsconfig.json	TypeScript compiler configuration.
tsconfig.spec.json	TypeScript configuration for testing.

7.2 React.JS Client

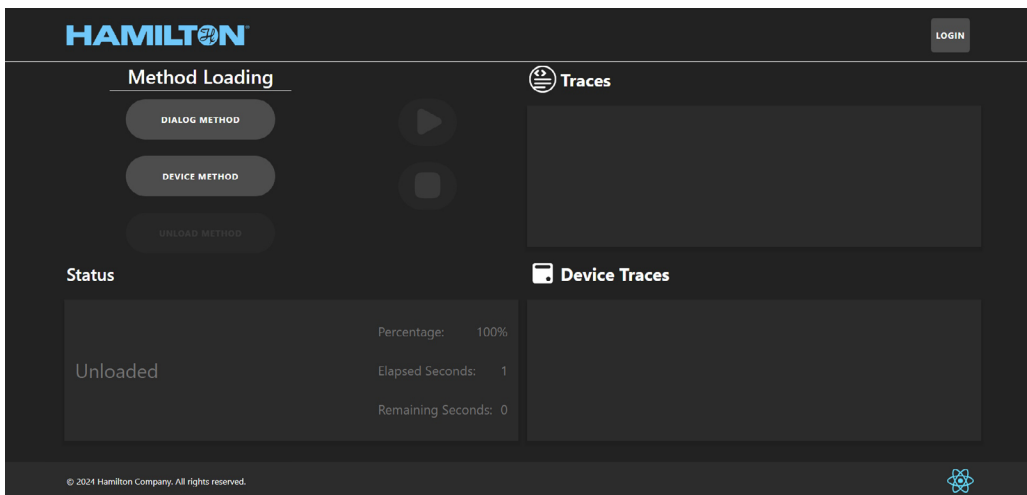


Figure 7-5: Snapshot of the home page.

The React sample client is built with the React.JS front-end library with Express.JS backend application to handle server-side functions.

For more information about the React.JS sample client, refer to the README.md markdown file in C:\Program Files (x86)\Hamilton\Bin\WebAPI\samples\React.

7.2.1 Setup

This section provides instructions on how to set up an environment to run the React.JS Client. The instructions provided are also included in the README.md guide under the project's source files.



Note: To run the example client, a set of pre-requisites are required. Refer to the markdown file in the project in C:\Program Files (x86)\Hamilton\Bin\WebAPI\samples\React to view the pre-requisites.

To launch the example client:

1. In a terminal or IDE, navigate to the directory of the React.JS sample client:
C:\Program Files (x86/)\Hamilton\Bin\WebAPI\samples\React.
2. Install the dependencies by running `npm install`.
3. Run `npm start`.

7.2.2 Features

This section provides descriptions of the features provided in the sample client.

7.2.2.1 Authentication and Authorization

If authentication on the machine is enabled, users are required to log in to an authorized account to perform calls to secured endpoints. The example client provides a feature to allow users to log in to their authorized account. For more information on secured endpoints and how to use them, refer to [Section 6.1](#) “How to use a Secured Endpoint”.

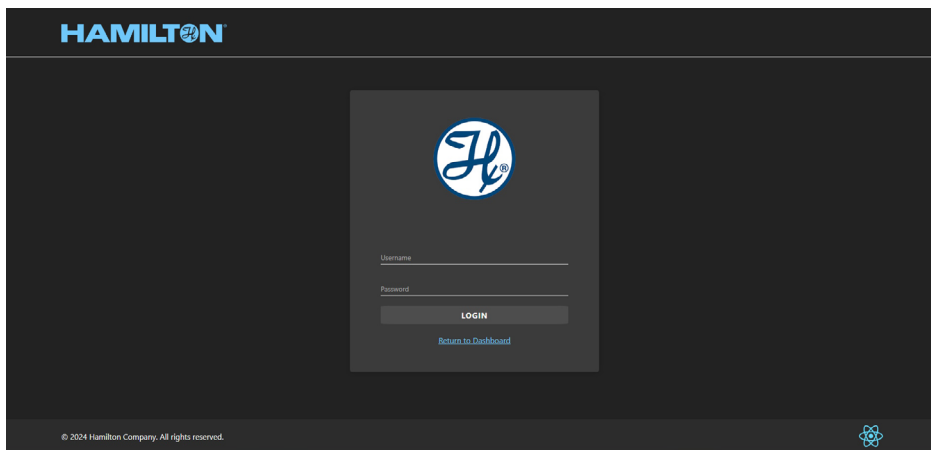


Figure 7-6: Snapshot of the Login page.

When starting the application, if the user is not signed in and authentication is enabled, the application launches on the Login page. Otherwise, it launches on the Home page.

When authentication is enabled, users must log in with the credentials of an authorized OS security group account to authenticate with the identity provider and gain authorized access to the dashboard executables and other functions in the VENUS software.

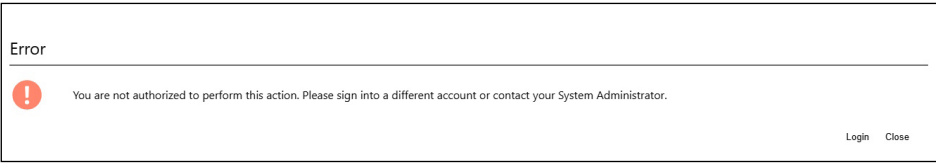


Figure 7-7: Snapshot of an error dialog for an unauthorized request.

Refer to [Section 5.1](#) “Operating System Security Groups” for information on the security groups used to grant access to functions in the VENUS software.

7.2.2.2 Executing Methods

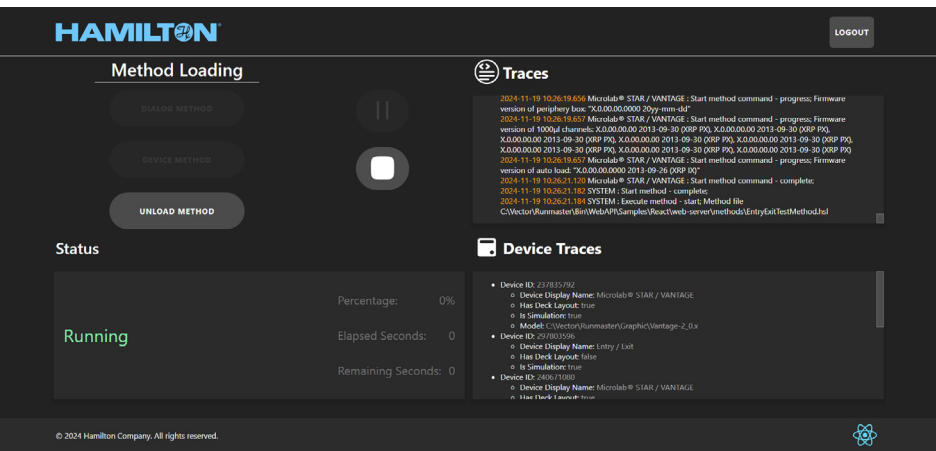


Figure 7-8: Snapshot of the sample client running the device method.

The sample client provides two methods that can be executed. One method provides a dialog prompt to the user, and the other method provides a simulated initialization procedure for an instrument.

When a method is running, the sample client updates the instrument's status, runtime traces, and if the method uses a device, device traces. Refer to [Section 6.2](#) “How to Run a Method” for more information on how to run a method and the operations that are available during the runtime.

7.2.3 Project Structure

Name	Description
public/	Contains static assets, HTML template, the manifest, and favicon
src/	Source files for the root-level application
src/__test__	Unit tests
src/components/auth	Button component for logging in and out
src/components/devicetracewindow	Window for device traces
src/components/footer	Footer component
src/components/header	Header component
src/components/loading-spinner	Loading spinner component to indicate loading states
src/components/modal	Modal component to display errors
src/components/run-executor	Contains button components using run-executor logic
src/components/runtracewindow	Window for runtime traces
src/components/statuswindow	Window for displaying runtime status
src/models	Models used to define entities, metrics, and events
src/pages	Home and login pages of the application
src/signalRSub	TypeScript based SignalR client
src/store	Redux.JS store
src/store/reducers	Redux.JS reducers
src/svgicons	SVG icons used in the application
web-server/build	Pre-compiled build of the node addon
web-server/methods	VENUS instrument methods
web-server/	Express.js app with a C++ node addon
.env	Environment variables for the application
package-lock.json	NPM dependency tree
package.json	NPM dependency list
ReactJS-Client-Architecture.png	Image of the project's architecture
README.md	Workspace documentation
tailwind.config.js	TailwindCSS configuration file



● Headquarters / Manufacturing



Years of Experience
75+



Locations Worldwide
22+



Employees Internationally
3,000+

To find a representative in
your area, please visit:
www.hamiltoncompany.com/contacts

Web: www.hamiltoncompany.com
USA: 800-648-5950
Europe: +40-356-635-055

Hamilton Americas & Pacific Rim
Hamilton Company Inc.
4970 Energy Way
Reno, Nevada 89502 USA
Tel: +1-775-858-3000
Fax: +1-775-856-7259
sales@hamiltoncompany.com

Hamilton Europe, Asia & Africa
Hamilton Central Europe S.R.L.
str. Hamilton no. 2-4
307210 Giarmata, Romania
Tel: +40-356-635-055
contact.hce.ro@hamilton-ce.com

